MPA (Multi-Proof Algorithm) Konsensüs Özellikleri

1. Giriş ve Arka Plan

Multi-Proof-Algorithm (MPA), ADAM işbirlikçi konsensüs çerçevesinin üzerine uygulanan hibrit bir konsensüs mekanizmasıdır. Düğüm (node) operatörlerini teşvik etmek, likiditeyi kilitlemek ve güvenli blok üretimini sağlamak için Proof-of-Stake (PoS) ile özel kriptografik kanıt mekanizmalarını birleştirir.

MPA, bir arada var olan dört staking kanıt türü sunar:

1. **Proof of Stake (PoS - Baseline)**: Standart coin yaşı tabanlı blok üretimi.
2. **Proof of Lock (PoL)**: CLTV veya CSV zaman kilitleri (timelocks) içeren çıktı betikleri (output scripts) kullanılarak yapılan staking olup, daha uzun kilit süreleri için ekstra madencilik ağırlığı (mining weight) kazandırır.
3. **Proof of Burn (PoB)**: Coin'lerin belirlenmiş harcanamaz bir adrese yakılması (burning) olup, zamanla doğrusal olarak azalan bir madencilik ağırlığı çarpanı sağlar.
4. **Proof of Masternode (PoM)**: Aktif Masternode teminatı ve bunun sürekli çalışma süresine (uptime) bağlı olarak madencilik ağırlığı tahsis edilmesi.

Bu ağırlık metrikleri, blok doğrulama sırasında zorluk hedeflerini (difficulty targets) doğrudan ölçeklendirerek, kaynak taahhüdünün doğrudan konsensüs gücüne dönüştüğü çok katmanlı bir konsensüs topolojisi oluşturur.

---

2. Konsensüs Mimarisi ve Parametreleri

MPA, `Consensus::UPGRADE_POMBL` ağ yükseltmesi (`vUpgrades[Consensus::UPGRADE_POMBL].nActivationHeight`) ile etkinleştirilir. Temel yapılandırması `src/consensus/params.h` içinde yer alır ve ağ başına `src/chainparams.cpp` içinde tanımlanır.

Temel Parametreler

- **`vUpgrades[Consensus::UPGRADE_POMBL].nActivationHeight`**: MPA konsensüs kurallarının etkinleştiği blok yüksekliği (`PoMBL`, **Proof of Masternode, Burn and Lock** anlamına gelir). Bu yüksekliğin altında ağ, eski kurallar veya Sürüm 11 ADAM kuralları altında çalışır.
- **`nTargetSpacing`**: Blok üretimi için hedef aralık (Mainnet ve Testnet üzerinde `30` saniye, Regtest üzerinde `10` saniye).
- **`mBurnAddresses`**: Kayıtlı harcanamaz yakma adreslerini (burn addresses) ve bunların aktif başlangıç yüksekliklerini içeren bir harita (map).

Ağ Etkinleştirme Yükseklikleri

Ağ	`UPGRADE_POMBL` Etkinleştirme Yüksekliği	Zorunlu Blok Sürümü
Mainnet	2000	Sürüm 12
Testnet	400	Sürüm 12

Ağ	`UPGRADE_POMBL` Etkinleştirme Yüksekliği	Zorunlu Blok Sürümü
Regtest	300	Sürüm 12

3. Madencilik Çekirdek Ağırlığı Hesaplamaları

Hibrit MPA sistemi altında bir sonraki bloğu kazma olasılığı, çekirdek değerlendirme (kernel evaluation) denklemi tarafından yönetilir (`src/kernel.cpp` içindeki `CStakeKernel::CheckKernelHash`):

$$\text{KernelHash} < \text{Target} \times \lfloor \text{Weight}_{\text{Total}} / 100 \rfloor$$

Verilen bir işlem çıktısı (UTXO) veya Masternode teminatı için madencilik $\text{Weight}_{\text{Total}}$ değeri, `src/kernel.cpp` içerisindeki `CalculateMPAWeight()` fonksiyonunda **1. PoM → 2. PoL → 3. PoB → 4. PoS** öncelik sırasıyla hesaplanır:

3.1. Proof of Stake (PoS - Baseline)

$W_{\text{PoS}} = \text{Amount}$

3.2. Proof of Lock (PoL)

Blok yüksekliği tabanlı mutlak (`OP_CHECKLOCKTIMEVERIFY`) veya göreceli (`OP_CHECKSEQUENCEVERIFY`) zaman kilitleri (timelocks) kullanılarak L_{MAX} değerine kadar kalan/göreceli L blok süresi boyunca kilitlenen işlem çıktılarına uygulanır:

$$W_{\text{PoL}} = \text{Amount} \times (1 + \gamma \cdot \min(\frac{L}{L_{\text{MAX}}}, 1.0))$$

- γ kilit çarpanı parametresidir (varsayılan: `2.0` , 3 kata kadar ağırlık bonusu verir).
- L_{MAX} değerlendirilen maksimum kilit süresidir (varsayılan: `50.000` blok).

3.3. Proof of Burn (PoB)

Standart (coinbase / coinstake olmayan: `!tx.IsCoinBase() && !tx.IsCoinStake()`) işlemlerle kayıtlı harcanamaz bir yakma adresine (Mainnet üzerinde `KTburnCYcDjF1GsXa5k3tc3eZrVVbqefyWr` / `KTburnUHdmKMVx4z8BcnZYZCWYKxRgfZp95` , Testnet üzerinde `ktBurn42LtQP2pJ2fS5X2kpRx4Sd86kNgx4` / `ktBurnKmNRzVXVVVDXxFhhKytkH5x2yqsCG`) gönüllü olarak gönderilen coin'ler, `mapAddressBurns` önbelleğinde yakma bloğundan bu yana geçen süre olan T (blok cinsinden) boyunca doğrusal olarak sıfıra düşen kişisel bir PoB staking ağırlık çarpanı alır. `CoinBase` veya `CoinStake` çıktılarındaki otomatik protokol yakımları (sahipsiz Model D koltuk payları) ise yalnızca `nMoneySupply` arzından kalıcı olarak düşülür ve kişisel PoB ağırlığı **oluşturmaz**:

$$W_{\text{PoB}} = \text{Amount} + \text{BurnAmount} \times \beta \times (1 - \frac{T}{T_{\text{MAX}}})$$

- β yakma teşvik çarpanıdır (varsayılan: `5.0`).
- T_{MAX} (`nBurnDecayBlocks`) azalma (decay) eşiğidir (Mainnet'te `500.000` blok, Testnet/Regtest'te `10.000` blok). $T \geq T_{\text{MAX}}$ olduğunda yakma bonusu 0'a düşer (geriye temel staking UTXO miktarı kalır).

3.4. Proof of Masternode (PoM)

Harcanmamış 4.200 KRISTA teminatı C ve aktif ömrü t_{active} olan aktif Masternode'lar (zincir üstü `mapMasternodeCollateralOutpoints` ve `IsMasternodeActiveAtHeight(..., 2880)` aracılığıyla `mapMasternodeActiveHeights` içindeki en erken aktif ping yüksekliği veya teminat UTXO yüksekliği kullanılarak %100 deterministik olarak hesaplanır; `mnodeman` yalnızca `chainActive.Height() <= 0` olan izole birim testlerinde kullanılır):

$$W_{\text{PoM}} = C \times (1 + \alpha \cdot \min(\frac{t_{\text{active}}}{T_{\text{MAX}}}, 1.0))$$

- α masternode ömür çarpanıdır (varsayılan: `1.0`, 2 kata kadar ağırlık sağlar).
- T_{MAX} (`nMasternodeUptimeLimit`) maksimum ömür olgunluğudur (Mainnet'te `100.000` blok, Testnet/Regtest'te `10.000` blok).
- Bir Masternode 2.880 blokluk aktivite penceresi içinde zincir üstü heartbeat ping göndermezse (veya `ENABLED` durumundan çıkarsa), PoM ağırlık bonusu devre dışı kalır (`t_active = 0`).

---

4. Uzun Ömürlü Masternode Kurulları (Long-Living Masternode Quorums - LLMQs)

Güvenli lider seçimi ve imza birleştirmeyi (signature aggregation) desteklemek için LLMQ'lar, **Hibrit BLS12-381 + secp256k1** kriptografik mimarisiyle (`CBLSSignedData` ve `CQuorumSignature::AggregateBLSSignatures`) uygulanır. Her kurul üyesi, kalıcı `secp256k1` Masternode kimlik anahtarıyla yetkilendirilmiş ve LevelDB'de sabitlenmiş (`PinBLSPubKey`) deterministik **BLS12-381** (`blst`) anahtarını (`DeriveBLSFromCKey`) kullanarak blok özetini (`blockHash`) imzalar. Doğrulama aşamasında (`CQuorumSignature::Verify`), üye kimlikleri ve sabitlenmiş BLS açık anahtarları doğrulandıktan sonra imzalayan üyelerin BLS12-381 açık anahtarları (`CBLSPubKey::Aggregate`) ve imzaları (`CBLSSignature::Aggregate`) birleştirilerek tek bir birleşik eşleşme (pairing) denklemiyle doğrulanır.

Kurul Seçimi ve Boyutu

- **Kurul Boyutu (Quorum Size)**: Tam olarak **5 üye**.
- **DKG Aralığı ve Blok Bazlı Canlılık Rotasyonu**: `GetActiveQuorum`, DKG oturumlarını Mainnet'te 1.400. bloğa kadar her **100 blokta** bir ve Testnet'te 1.000. bloğa / Regtest'te 800. bloğa kadar her **10 blokta** bir çalıştırır; bu yükseklikliklerden itibaren (Mainnet üzerindeki tüm `UPGRADE_POMBL >= 2.000` blokları dahil) `RunDKG(nHeight)` her blokta `((nHeight / dkgInterval) * dkgInterval) - 1` yüksekliğindeki deterministik dönem tohumunu kullanarak aktif kurul üyeliğini değerlendirir.
- **Aktif Masternode Filtreleme**: Herhangi bir ağda (Mainnet, Testnet veya Regtest) kodlanmış (hardcoded) anahtar kimliği filtrelemesi veya yerel cüzdan kısıtlaması uygulanmaz. Seçim, ağdaki tüm aktif ve etkinleştirilmiş Masternode'lar arasından dinamik olarak yapılır.
- **Belirleyici Geri Çekilme (Deterministic Fallback)**: 5'ten az aktif masternode mevcutsa, DKG oturum yöneticisi kayıtlı madenci havuzundan (`GetAdamMinerPool(nHeight - 1)`) üyeler seçmeye geri döner. Regtest'te kayıtlı madenci havuzu, yerel testleri kolaylaştırmak amacıyla **15 deterministik anahtarla** (0'dan 14'e kadar olan indekslerden türetilir) önceden doldurulur. Diğer ağlarda ise bu havuz, kayıtlı madencilerden (Coin-Lock veya PoW-Lock yoluyla) ve bootstrap madencilerinden oluşur.

İmza ve Eşik Doğrulaması

- Kurul üyeleri, `secp256k1` kimlik anahtarlarıyla yetkilendirilmiş deterministik **BLS12-381** (`DeriveBLSFromCKey`) anahtarlarını kullanarak blok özetini (block hash) imzalar ve `CQuorumSignature::AggregateBLSSignatures` ile imzalar birleştirilir.
- Sürüm `>= 12` olduğunda (`Consensus::UPGRADE_POMBL` yükseltmesiyle deterministik olarak — Mainnet'te 2.000, Testnet'te 400 ve Regtest'te 300 — Standart Mod aktifleştiğinde) kurul imzası `vQuorumSig` blok başlıklarına doldurulur. Model D (`UPGRADE_MODELD`), Mainnet'te 2.200, Testnet'te 500 ve Regtest'te 200 yüksekliğinde etkinleşen ayrı bir yükseltmedir.
- **Kurul Doğrulama Eşiği (`src/llmq.cpp` içindeki `CQuorumSignature::Verify`)**:
  - **Model D Aktivasyonundan Önce (Mainnet / Testnet üzerinde `!NetworkUpgradeActive(nHeight, UPGRADE_MODELD)`)**: Doğrulama eşiği **0 imzadır** (aktif LLMQ kurulları zorunlu hale gelmeden önce bootstrap sürecini kolaylaştırmak için doğrulama atlanır - bypassed).
  - **Model D Aktif Olduktan Sonra (Mainnet, Testnet ve Regtest)**: Hem blok üretimi (`miner.cpp`) hem de konsensüs doğrulaması (`llmq.cpp`), tüm ağlarda katı bir BFT salt çoğunluk eşiği olan `quorum.members.size() / 2 + 1` (5 üyeli standart bir kurul için **5'te 3 imza**) değerini zorunlu kılar.

> **NOT**
> **Eşik Değerleri Arasındaki Fark: ADAM İşbirlikçi Madencilik Eşiği** (madenci bulmaca çözümleri `vAdamSolutions` için uygulanan 11'de 7 kuralı) ile **LLMQ İmza Doğrulama Eşiği** (blok imzası `vQuorumSig` için Mainnet'te uygulanan 5'te 3 kuralı) karıştırılmamalıdır:
>
> - **ADAM Eşiği**: Seçilmiş 11 madenciden en az `GetAdamEffectiveThreshold` kadarının (`nAdamThresholdBootstrapLimit` altındaki başlangıç döneminde `5`, sonrasında Mainnet/Regtest üzerinde `nAdamThreshold = 7` ve Testnet üzerinde `3`; $t \geq 120$ sn durumunda zamana bağlı gevşetmeli) PoW bulmaca çözümünü (`vAdamSolutions`) çözmüş ve göndermiş olmasını şart koşar. Bu kural ADAM işbirlikçi bulmaca doğrulama katmanına aittir.
> - **LLMQ Eşiği**: Blok hash'inin geçerli sayılabilmesi için 5 LLMQ üyesinden en az 3'ünün hibrit BLS12-381 + secp256k1 anahtarlarıyla bloğu imzalamış olmasını (`vQuorumSig`) şart koşar. Bu kural merkeziyetsiz blok doğrulama katmanına aittir.

---

5. Konsensüs Dayatmaları ve Doğrulama Kuralları

Bir blok alındığında, `Consensus::UPGRADE_POMBL` aktif olduğunda `src/main.cpp` içerisindeki `CheckBlock()` fonksiyonundaki doğrulama kuralları aşağıdaki kontrolleri zorunlu kılar:

1. **Sürüm Zorunluluğu (Version Enforcement)**: Blok sürümü en az `12` olmalıdır. Sürümü 12'den küçük olan blok başlıkları `bad-version` kodu ile kesinlikle reddedilir.
2. **Yakma Çıktısı Bütünlüğü (Burn Output Integrity)**: Kayıtlı bir yakma adresinden harcama yapan tüm işlem girdileri, mempool katmanında (`AcceptToMemoryPool`) ve blok bağlantı katmanında (`ConnectBlock`) `bad-txns-spend-burn-address` hatasıyla kesinlikle reddedilir.
3. **Kilit Süresi Betiği Zorunluluğu (Locktime Script Enforcement)**: PoL için kullanılan kilitli işlem çıktıları kilitli betiklere kesinlikle uymalıdır ve belirtilen kilit yüksekliği/zaman damgası geçene kadar harcanamaz.

4. **Kurul Oyu Doğrulaması (Quorum Vote Validation)**: Blok başlığına dahil edilen kurul imzası (`vQuorumSig`), aktif LLMQ açık anahtarına (public key) karşı doğrulanır.

---

6. Test Etme ve Doğrulama

MPA konsensüs değişiklikleri çift taraflı bir test paketi ile tamamen kapsanmaktadır:

6.1. Birim Testleri (`test_kristatech`)

`src/test/mpa_tests.cpp` dosyasındaki C++ birim testleri (unit tests) şunları doğrular:

- Standart PoS için ağırlık hesaplamaları.
- Proof of Burn (PoB) için doğru azalma (decay) parametreleri ve azalma hesaplamaları.
- Proof of Lock (PoL) işlem girdileri için yükseklik ve süre hesaplamaları.

6.2. Fonksiyonel Testler (`consensus_pombl.py`)

Python fonksiyonel test paketi `test/functional/consensus_pombl.py` , kuralları yerel bir Regtest ağı üzerinde doğrular:

- Etkinleştirme yüksekliğine kadar blok üretimini simüle eder.
- Etkinleştirme sınırında sürümü 12'den küçük olan blokların reddedildiğini onaylar.
- Blok sürümü 12'nin kabul edildiğini onaylar.
- Bir yakma adresine fon gönderilmesinin kabul edildiğini doğrular.
- Bir yakma adresinden harcama yapma girişimlerinin engellendiğini ve `bad-txns-spend-burn-address` ile reddedildiğini doğrular.